RELATÓRIO TÉCNICO DE ENTREGA: AUTENTICAÇÃO E SEGURANÇA (2AV)

Disciplina: Projeto de Desenvolvimento de Software / API

Aplicação: Central de Comunicados Corporativos — ConnectPlus

Data de Entrega: 5 de novembro de 2026

Status da Entrega: Concluída

Integrantes da equipe: David William Dos Santos Guedes Alcoforado - 01926604

Júlio Silvestre de Lima Simão - 01888262

Maria Fernanda Barbosa Dos Santos - 01898944

Raquel Macêdo De França - 01910871

Sarah Leticia Souza Da Silva - 01910076

Ygor Da Silva Alves - 01902390

1. OBJETIVO DO PROJETO

O objetivo desta entrega foi implementar mecanismos robustos de autenticação de usuários, criptografia de credenciais e controle de acesso baseado em tokens na aplicação desenvolvida na

unidade anterior, garantindo a proteção de rotas críticas contra acessos não autorizados.

2. REQUISITOS MÍNIMOS OBRIGATÓRIOS IMPLEMENTADOS

2.1 Manutenção dos CRUDs da Primeira Unidade

A aplicação mantém a integridade de todas as rotas e regras de negócio desenvolvidas anteriormente. O CRUD da **Central de Comunicados** continua totalmente operacional, mapeado e integrado ao banco de dados relacional.

2.2 Cadastro e Provisionamento de Usuários com Credenciais

Foi estruturado o modelo relacional de usuários (`Usuario`) no arquivo `models.py`, permitindo o armazenamento seguro de identificadores, nomes, e-mails e perfis de acesso. O provisionamento inicial foi validado via script automatizado de banco de dados.

2.3 Armazenamento de Senhas Utilizando Hash Seguro

As senhas dos usuários **não são armazenadas em texto limpo**. Foi integrada a biblioteca de criptografia de alto nível `bcrypt` no arquivo `api/app/security.py`. A função `gerar_hash(senha: str)` intercepta a senha, gera um *salt* aleatório e calcula o hash seguro antes de persistir o registro no banco de dados SQLite (`connectplus.db`).

2.4 Autenticação por Login e Senha

Implementou-se o endpoint `POST /auth/login` consumindo o fluxo padrão `OAuth2PasswordBearer` do ecossistema FastAPI. O sistema valida o e-mail informado, localiza o registro correspondente e utiliza a função `verificar_senha(senha, senha_hash)` para comparar os hashes.

2.5 Geração de Token JWT com Tempo de Expiração

Após o login bem-sucedido, a aplicação utiliza a biblioteca `PyJWT` para assinar digitalmente um token **JSON Web Token (JWT)**. O token possui a alegação padrão `sub` (identificador do usuário) e a alegação `exp`, que define um tempo de expiração estrito de **60 minutos**, invalidando o token automaticamente após esse período.

2.6 Validação do Token JWT e Retorno 401 Unauthorized

Foi implementada uma camada de interceptação e injeção de dependências no arquivo `api/app/deps.py`. Todas as chamadas para rotas protegidas passam por uma validação que tenta decodificar o cabeçalho `Authorization: Bearer <TOKEN>`. Se o token estiver ausente, expirado ou corrompido, a API encerra a requisição imediatamente e retorna o código de status padrão **HTTP 401 Unauthorized** com o JSON `{"detail": "Not authenticated"}`.

2.7 Proteção de Rotas da Aplicação

O sistema superou a exigência mínima de duas rotas protegidas. Toda a estrutura do CRUD de comunicados (Publicar, Atualizar e Excluir), além do endpoint de identificação, exige obrigatoriamente a presença de um token de autenticação válido, identificando os métodos fechados através do ícone de cadeado na interface de documentação.

2.8 Endpoint para Consultar os Dados do Usuário Autenticado

Foi desenvolvido o endpoint `GET /auth/me`. Quando acionado com um token JWT válido, ele realiza a leitura do payload, valida o estado do usuário no banco de dados e retorna os dados de identificação (`id`, `nome`, `email`, `perfil`) do usuário atualmente logado com status **HTTP 200 OK**.

2.9 Configuração da Chave JWT via Variável de Ambiente

Para evitar a exposição de segredos no repositório de código fonte, a chave de assinatura simétrica (`SECRET_KEY`) e a URL do banco de dados (`DATABASE_URL`) foram desacopladas através da biblioteca padrão `os`. A aplicação consome esses valores dinamicamente do ambiente operacional por meio da função `os.getenv()`.

3. MAPEAMENTO TÉCNICO DAS EVIDÊNCIAS OBRIGATÓRIAS

As imagens anexadas no arquivo compactado comprovam a execução real de cada fluxo exigido pelo professor na interface automatizada do Swagger UI (`http://127.0.0`):

- **Evidência 1 — Swagger Geral e Mapeamento de Rotas (`1. Tela inicial.png` a `3. Tela inicial.png`):** Demonstra a interface oficial da API apresentando os novos endpoints de autenticação e os ícones visuais de bloqueio (cadeados abertos) nas rotas protegidas.
- **Evidência 2 — Bloqueio de Acesso Sem Autenticação (`4. Bloqueando acessos sem token válido (401).png` a `8. Bloqueando acessos sem token válido (401).png`):** Comprova a segurança do sistema. Ao disparar uma requisição para o endpoint `/auth/me` sem fornecer credenciais, a API nega o acesso e retorna com sucesso o código **HTTP 401 Unauthorized**.
- **Evidência 3 — Login e Emissão do JWT (`9. login com sucesso e emissão do JWT.png` a `12. Login com Sucesso e Emissão do JWT.png`):** Demonstra o preenchimento do formulário OAuth2 no botão *Authorize* global, o envio dos dados protegidos por hash, e a alteração dos cadeados visuais do sistema para o estado trancado (*Authorized*),

confirmando que a API devolveu o token Bearer JWT de forma correta.

- **Evidência 4 — Acesso Autorizado com Token Válido (`13. Evidência de login autorizado (200).png` a `17. Evidência de login autorizado (200).png`):** Demonstra o endpoint `/auth/me` injetando o cabeçalho técnico de autorização e respondendo com o status **HTTP 200 OK**, exibindo no corpo da resposta os dados do usuário autenticado no formato JSON.

4. INFORMAÇÕES DE ENTREGA DO GRUPO

Link do Repositório do GitHub: `https://github.com/SarahSouzaSilva/connectplus-auth`

- **Arquivo de Evidências Anexo:** `Entrega-Autenticacao e seguranca (Respostas).zip` (contendo as 17 capturas de tela validadas).